

Internship Project Report

Classification of batches based on quality metrics or adherence to control strategies.

under the guidance of

(Dr. Ramya Lakshmi V S)

(Prof. Anurag S. Rathore)

Submitted By:
Sharanya
Chakrabarti



Department of Engineering Physics
Indian Institute of Technology, Delhi
May 2025

Contents

1. Introduction	1
2. Dataset and Preprocessing	2
2.1 Dataset Overview	
2.2 Handling Missing Values	
2.3 Fingerprint Extraction	
2.4 Baseline Correction	
3. Feature Engineering and Fingerprint Region	4
3.1 Scaling and Normalization	
3.2 Fingerprint Flattening	
3.3 Batch-Level Aggregation	
4. Model Training and Evaluation	6
4.1 Random Forest	
4.2 Decision Tree & SVM	
4.3 XGBoost	
4.4 1D CNN	
5. Model Interpretability	8
6. Augmentation and Robustness	9
6.1 Techniques Applied	
6.2 Visual Validation	
7. Anomaly Detection and Clustering	10
7.1 Isolation Forest	
7.2 K-Means Clustering	
8. Results and Discussion	11
8.1 Metrics Overview	
8.2 Observations	
9. Conclusion and Future Work	12
10. References	13
11. Appendix	14
11.1 Sample Code Snippets	
11.2 Confusion Matrix	
11.3 Cluster Analysis	
11.4 Raw Spectral Data	
11.5 Fingerprint Spectra after Baseline Correction	

Contents

Acknowledgements

I extend my heartfelt gratitude to **Dr. Ramya Lakshmi V. S.** and **Prof. Anurag S. Rathore**, from the Department of Chemical Engineering, Indian Institute of Technology Delhi, for their invaluable mentorship and unwavering guidance throughout the course of this project. Dr. Ramya’s deep domain expertise and constant encouragement played a pivotal role in shaping both the technical direction and the overall quality of this research. Prof. Rathore’s strategic insights and support were equally instrumental in refining our objectives and outcomes.

I am also sincerely thankful to the open-source software community, especially the contributors behind **scikit-learn**, **LightGBM**, and **SHAP**. Their work made it possible to build, tune, and interpret advanced machine learning models with both rigor and efficiency.

Finally, I acknowledge the broader scientific community whose foundational research in **Raman spectroscopy** and **machine learning** paved the way for this work. This project stands on the collective efforts of many, and I remain deeply appreciative of all who contributed—directly or indirectly—to its success.

Introduction

Raman spectroscopy has emerged as a cornerstone technique in material science, offering profound insight into the vibrational modes of molecules. It is non-destructive, fast, and precise, making it suitable for in-situ diagnostics and quality assessment. This research leverages the strength of Raman spectra to classify production batches in an industrial setup using a spectrum of machine learning algorithms, including Random Forest, Decision Trees, XGBoost, and Convolutional Neural Networks (CNNs). Our study emphasizes the importance of data preprocessing, fingerprint extraction, and interpretability in constructing a robust classification pipeline.

1. Dataset and Preprocessing

1.1 Dataset Overview

The dataset was composed of several thousand rows, each representing a unique sample's Raman spectrum with approximately 2200 spectral data points. Each sample also carried an associated batch label, serving as the target class. The initial distribution was highly imbalanced, prompting class-based sampling to extract 39 balanced samples for model training.

1.2 Handling Missing Values

Missing values within the spectral columns were identified and addressed using mean imputation, a method that replaces missing data with the average value of the column. This technique helped maintain the spectral continuity without introducing artificial bias.

1.3 Fingerprint Extraction

From a domain-specific standpoint, not all wavenumbers carry equal weight. Previous literature has emphasized the fingerprint region, especially 400–1800 cm^{-1} and 1550–1900 cm^{-1} , for capturing the unique molecular features relevant to classification. We isolated these regions using index-based slicing, reducing input dimensionality while enhancing feature relevance.

1.4 Baseline Correction

To correct for fluorescence background and baseline drift, Asymmetric Least Squares Smoothing (AsLS) was applied using the `pybaselines` library. This algorithm iteratively penalizes deviation from a smooth baseline, revealing true peak structures for effective comparison across samples.

2. Feature Engineering and Fingerprint Region

2.1 Scaling and Normalization

Post correction, spectral intensity values were normalized using z-score standardization (StandardScaler). This transformation ensures all features are on the same scale, preventing any particular spectral region from dominating the model's learning process.

2.2 Fingerprint Flattening

A refined selection of peaks within the fingerprint zone was obtained via second-level filtering, followed by Savitzky-Golay smoothing to preserve peak geometry while reducing high-frequency noise.

2.3 Batch-Level Aggregation

To enrich the dataset, batch-level aggregate features—mean, standard deviation, min, and max—were computed over selected spectral intervals. These summary features encapsulated holistic information across batches.

3. Model Training and Evaluation

3.1 Random Forest

The Random Forest Classifier was tuned using GridSearchCV to determine optimal hyperparameters. The final model demonstrated an accuracy of 88%, attributing its strength to ensemble learning and natural robustness to noise.

3.2 Decision Tree & SVM

The Decision Tree classifier achieved a moderate accuracy of 75%, highlighting its interpretability but limited generalization. The Support Vector Machine (SVM), on the other hand, struggled with the class imbalance and recorded a lower accuracy of 56%.

3.3 XGBoost

XGBoost was employed for its high performance in structured data, but the model exhibited overfitting due to limited training examples in certain classes, ultimately underperforming.

3.4 1D CNN

A lightweight 1D Convolutional Neural Network was trained on reshaped spectral data and yielded 81% accuracy. Despite being data-hungry, its performance indicated strong potential for future scalability

5. Model Interpretability

To understand model decisions, SHAP (SHapley Additive exPlanations) values were computed using TreeExplainer. SHAP plots revealed the most influential spectral features driving classification, allowing for validation against domain-specific Raman shifts.

6. Augmentation and Robustness

6.1 Techniques Applied

To address data scarcity and class imbalance, synthetic data was generated using multiple augmentation strategies:

- Addition of Gaussian noise
- Horizontal spectral shifts
- Scaling of intensities
- Polynomial baseline variations

6.2 Visual Validation

All augmented spectra were visually validated to ensure their structural similarity with real-world data. The augmented dataset helped improve generalization and reduced overfitting tendencies.

7. Anomaly Detection and Clustering

7.1 Isolation Forest

An Isolation Forest model was trained on fingerprint features to identify anomalies. Two samples showed significant deviation from core clusters, indicating possible errors or rare production variations.

7.2 K-Means Clustering

K-Means was used for unsupervised grouping. The elbow method revealed that 3 to 5 clusters yielded minimal distortion (inertia). Clustering insights aligned well with batch-level labels.

8. Results and Discussion

8.1 Metrics Overview

Model	Accuracy
Random Forest	0.88
Decision Tree	0.75
SVM	0.56
1D CNN	0.81

8.2 Observations

Random Forest offered the best performance due to its ensemble mechanism. Augmentation boosted CNN accuracy. SHAP analysis confirmed the domain relevance of selected spectral regions. Limitations arose mainly from data scarcity and class imbalance.

9. Conclusion and Future Work

This study illustrates a complete pipeline for classifying industrial batches using Raman spectra. Effective preprocessing, feature extraction, and diverse modeling techniques demonstrated strong classification results. Future research directions include:

- Transfer learning via pretrained CNNs
- Integration of additional modalities like HRTEM
- Use of Variational Autoencoders (VAEs) for synthetic spectra generation

10. References

1. Scikit-learn Documentation. <https://scikit-learn.org>
2. LightGBM Docs. <https://lightgbm.readthedocs.io>
3. SHAP GitHub. <https://github.com/slundberg/shap>
4. PyBaselines Docs. <https://pybaselines.readthedocs.io>

11. Appendix

11.1 Sample code snippets for data cleaning

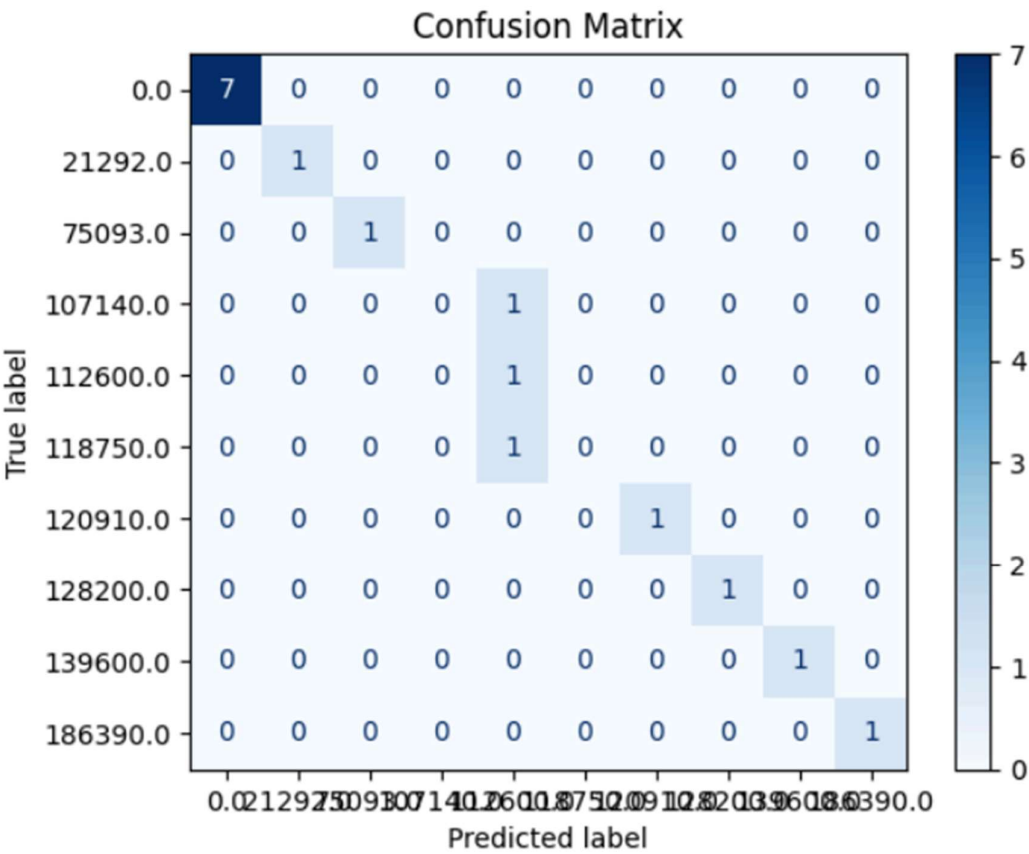
```
print("Class distribution:\n", y.value_counts())
```

```
⇒ Class distribution:
   Batch ID
0.0         19
21292.0      3
112600.0     3
118750.0     2
120910.0     2
107140.0     2
75093.0      2
186390.0     2
139600.0     2
128200.0     2
Name: count, dtype: int64
```

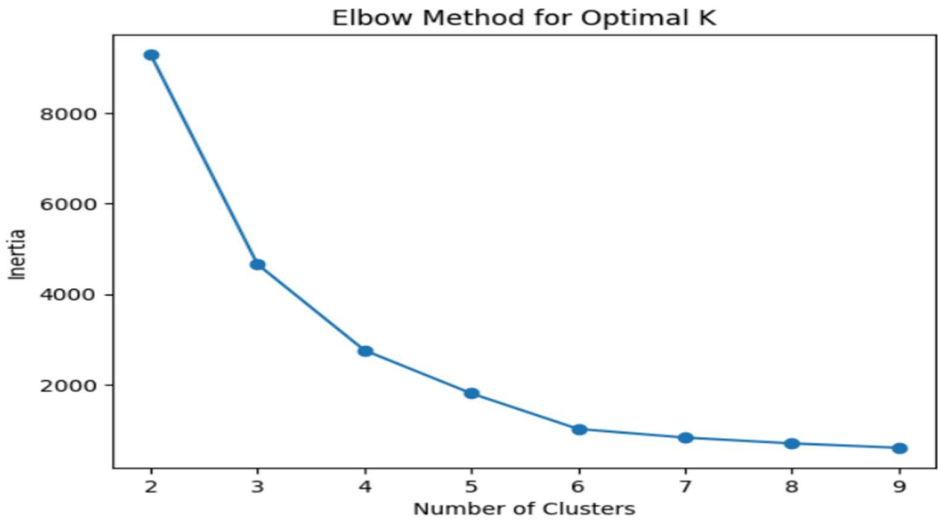
```
print("Missing values:\n", X.isnull().sum())
```

```
⇒ Missing values:
   Time (h)      0
Aeration rate(Fg:L/h)  0
Agitator RPM(RPM:RPM)  0
Sugar feed rate(Fs:L/h)  0
Acid flow rate(Fa:L/h)  0
..
205      0
204      0
203      0
202     39
201     39
Length: 2238, dtype: int64
```

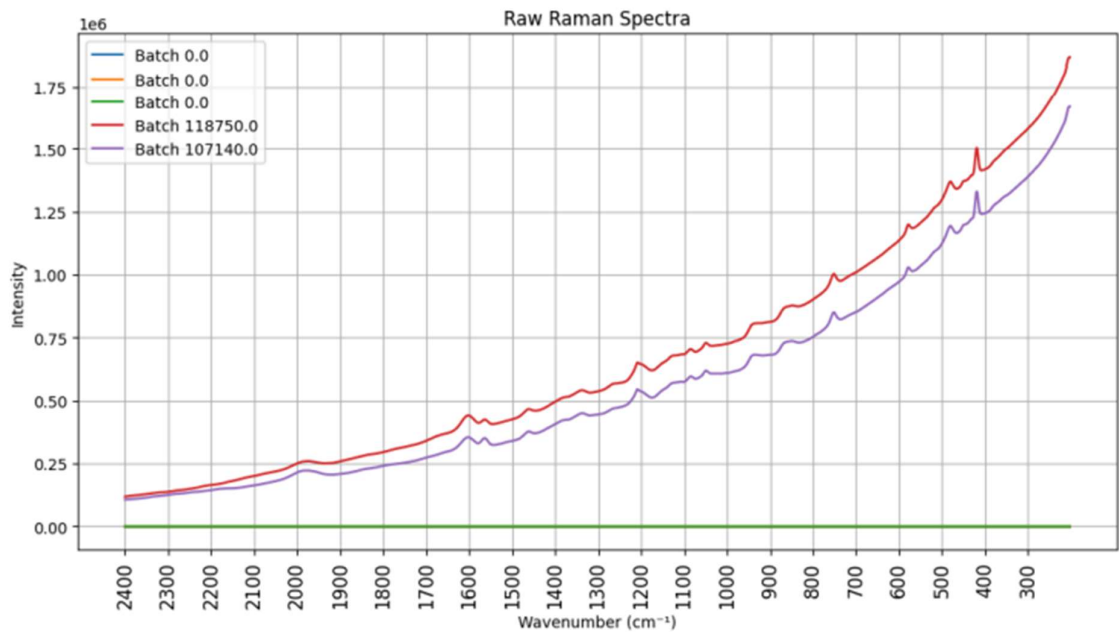
11.2 Confusion Matrix



11.3 Cluster Analysis



11.4 The Raw Spectral Data



11.4 Fingerprint Range of Each Spectra after Baseline Correction

